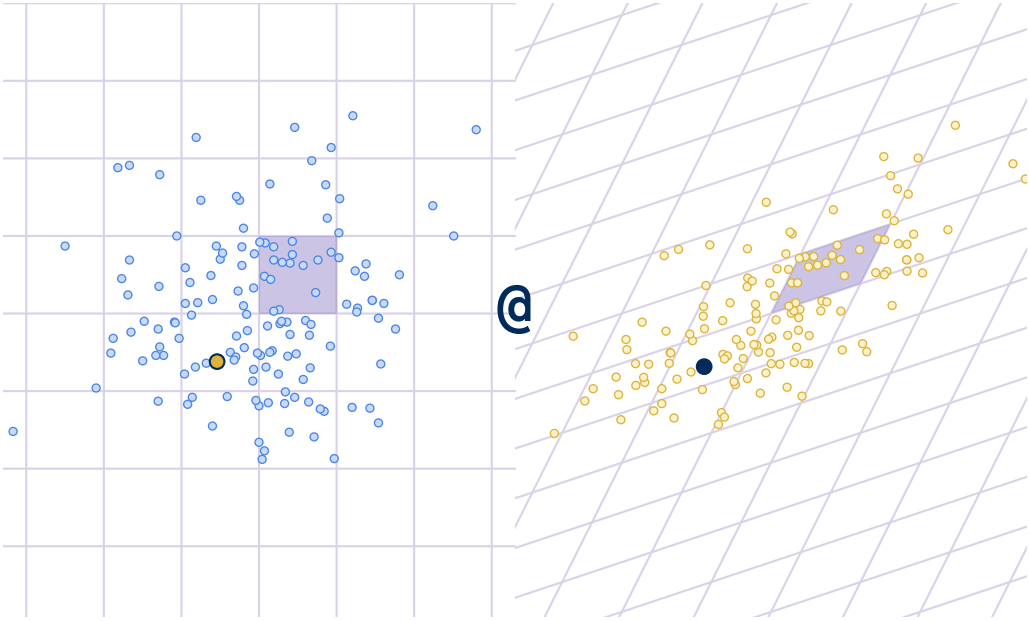


UNIT 01 · CHAPTER 1

High Dimensional Data

CSE 10124 · Building ChatGPT
William Theisen · University of Notre Dame

Representations

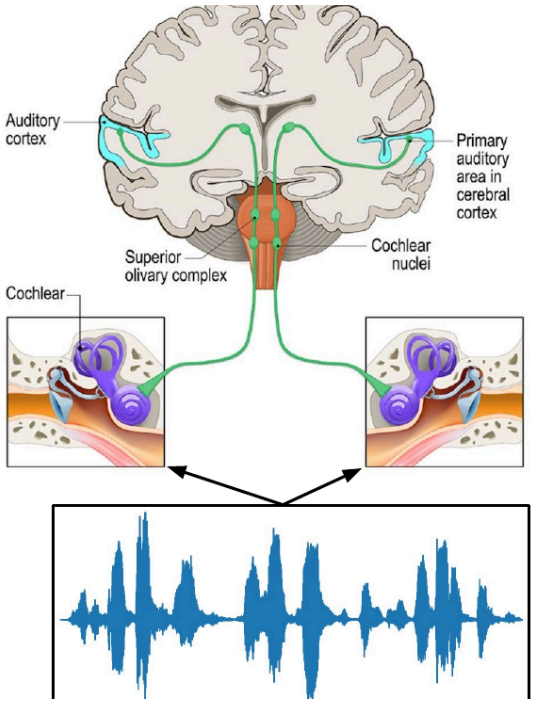
By the end of this course we will have built a small version of ChatGPT from scratch. Unfortunately, a computer can't read a sentence, look at a picture or watch a football game. All it can do is arithmetic on numbers, so **before any model can learn anything, the thing it is learning about has to become numbers**. That turns out to be one of the more important decisions in machine learning, and it is where we'll start.

Take the second game of Notre Dame's 2012 season, Purdue at Notre Dame Stadium. There were `80,795` people in the stands, two hours of plays, a band, the weather, and a final score of `20` to `17`. The scoreboard keeps exactly one part of all that. It tells us who won, and nothing at all about what it was like to be there.

A **representation** is a way of describing something by recording selected information about it. The score, a written recap and a photograph are three different representations of the same game, and each one keeps some things and throws the rest away. Which one is "right" depends entirely on the question we want to answer.

Representations in the Brain

Our brains do the same thing. Your ears don't send sound to your brain, they turn the vibrations of the air into patterns of nerve signals, which travel up to the auditory cortex on each side of your head (pictured below). Your eyes do the same with light. **Everything you know about the world arrives as a representation built by your senses**, and, much like a scoreboard, those representations leave a lot out.



Features

Our dataset has every game Notre Dame played from `2012` through `2022`, and here are the first five, with seven of its columns:

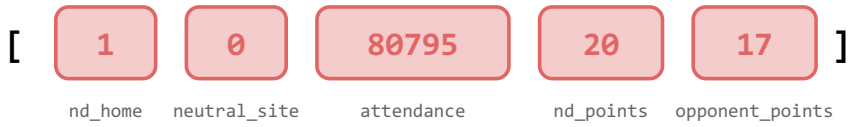
Opponent	ND home	Neutral site	Attendance	ND points	Opp. points	Win
Navy	0	1	48,820	50	10	1
Purdue	1	0	80,795	20	17	1
Michigan State	0	0	79,219	20	3	1
Michigan	1	0	80,795	13	6	1
Miami	1	0	62,871	41	3	1

Each column records one thing about a game, like how many people came or how many points Notre Dame scored. One measured property like this is called a **feature**. Notice that every feature here is a number, even where the property isn't really a quantity: ND home is `1` if Notre Dame was the home team and `0` if it wasn't (the Navy game was played in Dublin, Ireland, hence the neutral site).

Each row is one game. One represented item like this is called a **data point**, or more often a **sample**. Choosing which columns to keep is choosing our representation: a model that only ever sees these seven columns has no idea it rained, or who the quarterback was.

Feature Vectors

To a computer, a sample is just its row of numbers in a fixed order. Leaving out the opponent and the win for now, the Purdue game becomes five numbers:



An ordered list of feature values for one sample is called a **feature vector**. The order is what gives each number its meaning. The `20` means Notre Dame's points only because it is in the fourth position, and if we swapped the last two numbers, `[1, 0, 80795, 17, 20]` would describe a game Notre Dame lost. So **every sample has to use the same features in the same order**, or comparing them is meaningless.

The collection of all our feature vectors is our **dataset**. Stacked on top of each other they make a table again, one row per sample and one column per feature, which is exactly the spreadsheet we started with. Ours has `141` rows, one for every game.

Our Dataset in Code

In Python the whole spreadsheet loads in one line with pandas. To do any math on a row, we turn it into a **tensor**, the kind of list of numbers that PyTorch (the library we'll build our models with) works with:

```python
import pandas as pd
import torch

games = pd.read_csv("nd_football.csv")
print(games.shape)

cols = ["nd_home", "neutral_site", "attendance",
        "nd_points", "opponent_points"]
x = torch.tensor(games.loc[1, cols].to_numpy(dtype=float))
print(x.tolist())
print(x.shape)
```

OUTPUT

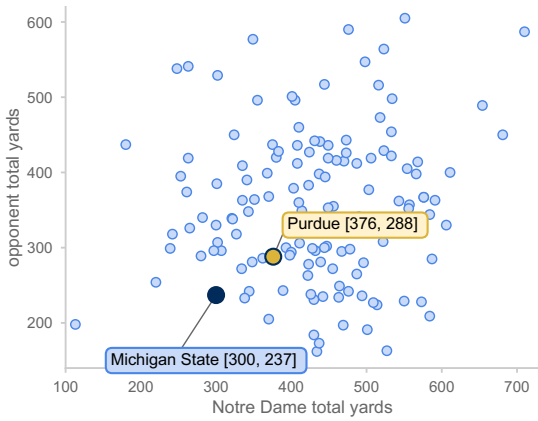
```
(141, 45)
[1.0, 0.0, 80795.0, 20.0, 17.0]
torch.Size([5])
```

The shape says we have `141` samples and `45` columns. Python counts rows from `0`, so row `1` is the Purdue game, and it comes back as the same feature vector as on the last page. A tensor's `shape` tells us how many numbers it holds along each direction, here just `5`.

Notice that the computer has no idea what any of these numbers mean. To it, `80795` is just a big number that happens to be third. **Whatever we want a model to learn, it has to find in the numbers we give it.**

Dimensionality

If we keep just two features, the yards Notre Dame gained and the yards its opponent gained, every game becomes a point on a plane. Purdue's feature vector, `[376, 288]`, is the point `376` across and `288` up:



Each feature is one axis. Adding a third feature, say Notre Dame's turnovers, would lift every game off the page into three dimensions, and Purdue would become the point `[376, 288, 1]`. The number of features a sample has is called its **dimensionality**.

Nothing stops us at three. If we used all `40` numeric columns in our spreadsheet, every game would be a point in `40`-dimensional space. Unfortunately nobody can draw that, but **every bit of math we'll do works exactly the same way** in `40` dimensions as it does in two.

High Dimensional Data

Forty dimensions is actually pretty modest. Data where every sample has many features is called **high dimensional data**, and most of the data that modern AI works with is very high dimensional.

A small black and white picture, `28` pixels by `28`, is one number per pixel (how bright it is). Read row by row, that is a feature vector with `784` dimensions, which is exactly how we'll hand pictures of handwritten digits to our first model in Programming Day 01. A photo off your phone has tens of millions.

Text gets the same treatment. The model we'll build in this course, irishQwen, represents every piece of a word as a list of `1,024` numbers, and a whole sentence is a stack of those. **ChatGPT never sees words, only very long feature vectors**, and a good part of this course is about how words become those vectors in the first place.

We'll stay with our football games for the rest of this chapter, since a box score is high dimensional enough to show every idea and small enough to check by hand.

Tasks and Targets

So far we've treated every column the same, but one of them is special. The win column is the thing we would actually like to know about a game. What we're trying to achieve with a model (here, predicting whether Notre Dame won) is called the **task**, and the output the model is trying to predict is called the **target**.

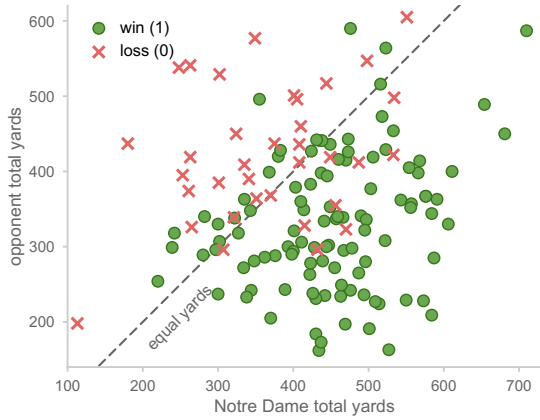
Our spreadsheet has the target filled in for every game. So for any game we can ask a model for its guess, check it against what really happened, and **tell the model whether it guessed correctly**. These correct answers are called the **ground truth**, and learning from a dataset that includes them is called **supervised learning**.

Not every dataset is this lucky. Somebody had to write down who won each game, and for a lot of problems (which emails are spam, which X-rays show a fracture) that somebody is a person labeling samples by hand, which is slow and expensive.

Text is a happy exception. If the task is predicting the next word of a sentence, the ground truth is just the word that really came next, and every book ever written is already labeled. That is a big part of why language models could be trained on so much data.

Classification Tasks

Our target only has two possible values, a win (`1`) or a loss (`0`). When the target is a category or label like this, we call it a **classification task**. Notre Dame won `106` of its `141` games and lost `35`.



Pictured above is the same plane as before, with every game marked by its label. Most of the wins sit below the dashed line, where Notre Dame gained more yards than its opponent. In the `96` games where it did, Notre Dame won `86`, and in the other `45` it won only `20`.

Suppose a new game lands somewhere on this plane. If every game around it was a win, it's a pretty safe bet that this one was too. That only works if we can say which games are "around" it, which means **measuring how far apart two games are**.

Euclidean Distance

Take two games from 2012, Purdue (376 yards gained, 288 allowed) and Michigan State (300 and 237). On our plane Michigan State is 76 yards to the left of Purdue and 51 below it. The straight line between them is the long side of a right triangle, so Pythagoras gives us its length:

two games p and q, two features each: square the differences, add, take the square root

$$d = \sqrt{(p_1 - q_1)^2 + (p_2 - q_2)^2}$$

Purdue and Michigan State

$$= \sqrt{76^2 + 51^2} = \sqrt{8377} \approx 91.5$$

This straight-line distance between two feature vectors is called the **Euclidean distance**. With more features we just keep adding squared differences, so the same formula works for any two vectors p and q with any number of features n:

[1.1]

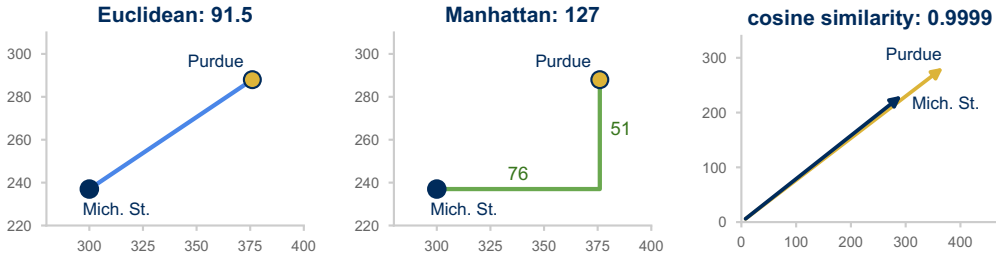
square each difference, add them up, take the square root

$$d(p, q) = \sqrt{\sum_{j=1}^{n} (p_j - q_j)^2}$$

The Σ just means "add up", one squared difference for each feature from the first ($j = 1$) to the last ($j = n$). In 40 dimensions that is 40 squared differences, tedious by hand and trivial for a computer.

Other Distance Metrics

Euclidean distance isn't the only option. Any rule for measuring the distance or similarity between two feature vectors is called a **metric**, and two others come up all the time. Pictured below are all three, measured between the same two games.



The **Manhattan distance** adds up the absolute differences instead of squaring them (in the middle). It's the distance you'd walk between two points if you had to stick to a city grid, here `76` + `51` = `127` yards. The vertical bars mean "the size of", so a difference of `-51` counts as `51`.

[1.2]

add up the absolute differences

$$d(p, q) = \sum_{j=1}^{n} |p_j - q_j|$$

The third panel is a different kind of measure entirely, and needs a page of its own.

Distances in Code

With PyTorch both distances are a line each. Subtracting two tensors subtracts them one feature at a time, so `diff` holds both differences at once, and `** 2`, `abs` and `sum` work on every number in it:

```python
import torch

purdue = torch.tensor([376., 288.])
msu = torch.tensor([300., 237.])

diff = purdue - msu
print(diff)
print(torch.sqrt((diff ** 2).sum()))
print(torch.abs(diff).sum())
```

OUTPUT

```
tensor([76., 51.])
tensor(91.5260)
tensor(127.)
```

The last two lines are the two formulas from the last two pages, written out almost word for word. Nothing in them cares how many features there are, so the same three lines would compare two games with `40` features, or two pictures with `784`.

Doing the same operation to every number in a tensor at once, instead of writing a loop, is how PyTorch code is almost always written. It is shorter, and on a GPU it is a great deal faster.

Cosine Similarity

The third metric, **cosine similarity**, ignores how far out each game is and only asks whether the two point in the same direction from `(0, 0)`. It is the cosine of the angle between them, so `1` means exactly the same direction and `0` means at right angles. To compute it we need the **dot product**: multiply the two vectors feature by feature and add up the results.

multiply matching features, then add

[1.3] $$p \cdot q = \sum_{j=1}^{n} p_j q_j \qquad 376 \cdot 300 + 288 \cdot 237 = 181056$$

the dot product, divided by both lengths

[1.4] $$\cos\theta = \frac{p \cdot q}{\|p\| \, \|q\|}$$

Here $\|p\|$ is the length of p, its Euclidean distance from `(0, 0)`. Dividing by both lengths is what makes cosine similarity ignore how far out each game is. Double every number in a game and its dot product with any other game doubles, but so does its length, and the two cancel.

Cosine Similarity in Code

In PyTorch, `@` between two vectors is the dot product, and `torch.norm` gives a vector's length. Here is Purdue against Michigan State, and against Wake Forest:

```
import torch

purdue = torch.tensor([376., 288.])
msu = torch.tensor([300., 237.])
wake = torch.tensor([584., 209.])

def cos_sim(a, b):
    return a @ b / (torch.norm(a) * torch.norm(b))

print(purdue @ msu)
print(cos_sim(purdue, msu))
print(cos_sim(purdue, wake))
```

OUTPUT

```
tensor(181056.)
tensor(0.9999)
tensor(0.9523)
```

Purdue and Michigan State point almost exactly the same way, because Notre Dame outgained both by about the same ratio. Wake Forest (`584` yards to `209`) sits about `18` degrees away. Cosine similarity is the usual way to compare words once they are vectors, and we'll be using the dot product a great deal.

Scaling

So far we've only used yards. Let's add a third feature, attendance, and measure Euclidean distances from Purdue again:

2012 game	ND yards	Opp. yards	Attendance	Distance from Purdue
Purdue	376	288	80,795	
Michigan State	300	237	79,219	1,578.7
Wake Forest	584	209	80,795	222.5

Wake Forest is now much "closer" to Purdue than Michigan State is, even though Notre Dame won the Wake Forest game `38` to `0` and only just got past Purdue `20` to `17`. The only reason is the crowd. Both were home games in front of a full Notre Dame Stadium, the Michigan State crowd was `1,576` smaller, and a difference of `1,576` people swamps a difference of `76` yards.

Attendance in our data runs from `3,535` (North Carolina in 2020, when crowds were capped during COVID) to `115,109` (at Michigan in 2013), while yards only run from `113` to `710`. In raw units, **every distance between our games is really just a distance in attendance**. Changing the range of our numerical features so that their units don't dominate a comparison (or a model) is called **scaling**.

Normalization and Standardization

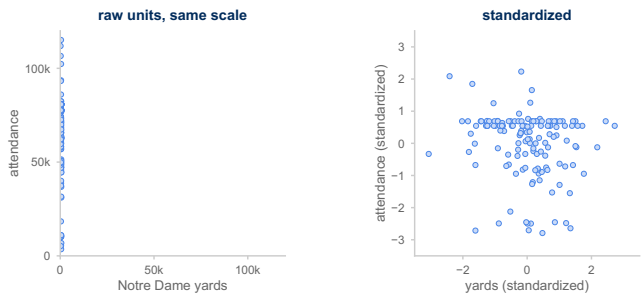
There are two common ways to scale a feature. **Normalization** squeezes each feature into the range `0` to `1`, by subtracting the smallest value and dividing by the range. **Standardization** subtracts the feature's mean μ and divides by its standard deviation σ (roughly, how far a typical value sits from the mean), so every feature ends up centered on `0` with a spread of `1`.

[1.5] normalization, to the range 0 to 1

$$x' = \frac{x - \min(x)}{\max(x) - \min(x)}$$

[1.6] standardization, mean 0 and standard deviation 1

$$x' = \frac{x - \mu}{\sigma}$$



On the left, in raw units drawn to the same scale on both axes, the `139` games with a recorded crowd are squashed into a thin strip, because next to attendance there is hardly any room for yards. Standardized (on the right), **both features get a say**.

Standardization in Code

Standardizing a whole dataset is one line, since `mean(dim=0)` and `std(dim=0)` work down each column (each feature) separately. Here are the distances from Purdue again, raw and then standardized:

```
import pandas as pd
import torch

games = pd.read_csv("nd_football.csv")
cols = ["nd_total_yards", "opp_total_yards",
        "attendance"]
X = torch.tensor(games[cols].dropna().to_numpy(),
                 dtype=torch.float32)
Z = (X - X.mean(dim=0)) / X.std(dim=0)

# rows 1, 2 and 10: Purdue, Michigan State, Wake Forest
for name, X_ in [("raw", X), ("standardized", Z)]:
    d_msu = torch.sqrt(((X_[1] - X_[2]) ** 2).sum())
    d_wake = torch.sqrt(((X_[1] - X_[10]) ** 2).sum())
    print(f"{name}: {d_msu:.2f} {d_wake:.2f}")
```

OUTPUT

```
raw: 1578.66 222.50
standardized: 0.91 2.17
```

Once every feature is standardized, Michigan State is back to being the closer game (`0.91` against `2.17`). The distances are no longer in yards or people, they're in standard deviations, which is a strange unit but at least the same one for every feature. (`dropna` drops the two games with no recorded crowd, both later than row `10`, so our three games keep their rows.)

Numerical and Binary Features

Scaling only makes sense for features that are numbers in the first place. Most of our columns hold a **numerical feature**, a measured property represented as a number, and these come in two kinds.

A **continuous feature** is measured along a scale and can include decimal values. Notre Dame held the ball for `34.23` minutes against Purdue, and it could just as easily have been `34.24`. A **discrete feature** is made of separate, countable values, usually whole numbers. Notre Dame turned the ball over once against Purdue, and nobody has ever turned it over `1.5` times.

Some columns are numbers in only the loosest sense. ND home is `1` or `0`, and a feature with two possible values like this is a **binary feature** (our target, win or loss, is one too). Notice that it takes two binary columns, ND home and neutral site, to say where a game was played. Notre Dame played `70` games at home, `25` at neutral sites and `46` away.

That leaves the opponent column, which isn't a number at all.

Categorical Features

Notre Dame played `51` different opponents in these eleven seasons, Navy, Stanford and USC the most (`10` times each). The opponent is a **categorical feature**: its values are categories rather than measurements.

The obvious move is to number them, Navy as `1`, Purdue as `2`, Michigan State as `3` and so on (this is called label encoding). Unfortunately, a distance metric will take those numbers at their word. Navy would be `1` away from Purdue and `2` away from Michigan State, which is purely an accident of the order we wrote them down in. Opponents have **no inherent order**, and categories like this are called **nominal**.

Some categories do have an order. A player's class year goes freshman, sophomore, junior, senior, and a senior really is "further" from a freshman than a sophomore is. Categories with a meaningful order are called **ordinal**, although the gap between each step still isn't necessarily equal or measurable.

One-Hot Encoding

For nominal features the standard fix is **one-hot encoding**: give every category its own binary feature, set to `1` for the category the sample belongs to and `0` for all the others.

2012 game	Navy	Purdue	Michigan State	…
Navy	1	0	0	…
Purdue	0	1	0	…
Michigan State	0	0	1	…

Now any two different opponents are exactly the same distance apart ($\sqrt{2}$, for the record), which is what we wanted. Unfortunately, every category costs us a whole feature, so our one opponent column would become `51` columns, almost all of them `0`.

One-Hot Encoding in Code

PyTorch will do the encoding once each category has a number. pandas gives every opponent a code from `0` to `50` (in alphabetical order), and `one_hot` turns each code into a row of `51`:

```
import pandas as pd
import torch

games = pd.read_csv("nd_football.csv")
opp = games["opponent"].astype("category")
codes = torch.tensor(opp.cat.codes.to_numpy(),
                     dtype=torch.long)
onehot = torch.nn.functional.one_hot(codes)

print(opp.cat.categories.size)
print(onehot.shape)
k = onehot[1].argmax().item()
print(onehot[1].sum().item(), k, opp.cat.categories[k])
```

OUTPUT

```
51
torch.Size([141, 51])
1 34 Purdue Boilermakers
```

Purdue's row has a single `1`, in position `34`, and zeros everywhere else. Notice that our target can be written the same way: a loss is `[1, 0]` and a win is `[0, 1]`, one position per class. That will turn out to be very handy.

Target Leakage

With every column turned into numbers, it's tempting to hand a model all of them. Unfortunately, some of them give the answer away. The margin column is Notre Dame's points minus the opponent's, and in all 141 games the margin is positive exactly when Notre Dame won. A model given the margin can be right every time by learning one rule, and **it would be completely useless**, because the day we know the margin we already know who won.

Information in an input feature that directly or indirectly reveals the target is called **target leakage**. The margin (or both teams' points) is the direct kind. The indirect kind is sneakier, a column that was recorded after the fact and just happens to track the result. The fix is to ask of every feature whether we would actually know it at the moment we make the prediction, and drop it if we wouldn't.

From here on we'll describe a game by its yards and turnovers instead of its points. They don't decide the result the way points do, but they are also only known once the game is over, so a model built on them can tell us which games look like wins, not call a game before kickoff. That is fine for our purposes, and worth keeping in mind.

Changing Dimensionality

Let's describe each game with four features: the yards Notre Dame gained and the yards it allowed (both in hundreds, so `376` yards is `3.76`), and each team's turnovers. What we want out is the answer, and on the One-Hot Encoding page we saw that the answer can be written as two numbers, one for a loss and one for a win. So a model has to take in a vector of `4` numbers and hand back a vector of `2`. **It has to change the dimensionality of our data.**

The tool for that is the **matrix**, a grid of numbers. Multiplying a matrix by a vector gives back a new vector, and the simplest example is a `2` by `2` matrix A times the vector `[1, 1]`:

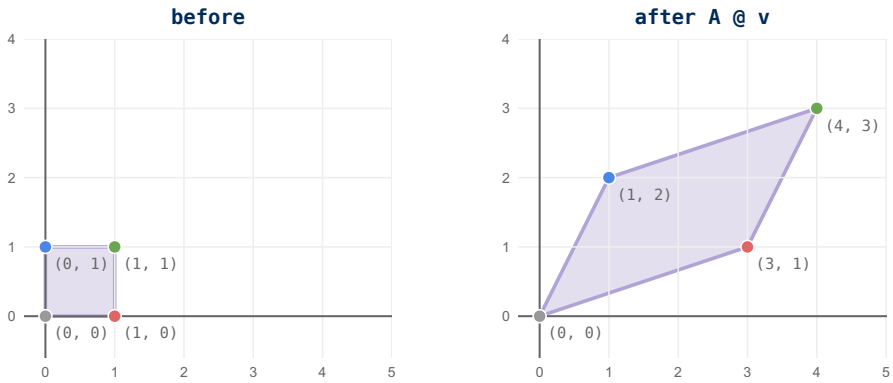
each output is one row of A dotted with the vector

$$A\,v = \begin{bmatrix} 3 & 1 \\ 1 & 2 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 3 \cdot 1 + 1 \cdot 1 \\ 1 \cdot 1 + 2 \cdot 1 \end{bmatrix} = \begin{bmatrix} 4 \\ 3 \end{bmatrix}$$

Each number in the answer is just a dot product, the multiply-and-add from Cosine Similarity, between one row of A and the vector. Two rows give two numbers out. A matrix with two rows and four columns would take in four numbers and still give back two, which is exactly the change we want.

Linear Transformations

It helps to see what a matrix does to a whole set of points. Pictured below, the four corners of a square, `(0, 0)`, `(1, 0)`, `(0, 1)` and `(1, 1)`, are each multiplied by the same matrix A from the last page.



`(1, 0)` lands on `(3, 1)`, `(0, 1)` on `(1, 2)`, and `(1, 1)` on `(4, 3)`, the answer from the last page. The square is stretched and tilted into a parallelogram, but straight lines stay straight, lines that were parallel stay parallel and `(0, 0)` stays put. A change like this, made by multiplying by a matrix, is called a **linear transformation**.

We can think of a linear transformation as **creating a new representation of our data**. Every point is still there, just described by different numbers, and the right matrix can move points that were hard to tell apart into places where they are easy to tell apart.

Matrix Multiplication

Our games are written as rows, so we'll put the game first and the matrix second. The rule doesn't change: **every row of the first meets every column of the second**, and each meeting is one dot product. We write the matrix product with `@`, as Python does. A game with `4` features is `<1, 4>` (one row, four columns), and a matrix with `4` rows and `2` columns gives back `<1, 2>`:



The two inner numbers (the `4`s) have to match, since each dot product pairs the game's four features with four numbers from one column. The outer numbers give the shape of the answer.

A Hand-Picked Matrix

Here is the Purdue game, `[3.76, 2.88, 1, 2]`, multiplied by a matrix W that we picked by hand. Each column of W makes one output, so the first column produces a loss score and the second a win score:

Purdue: yards gained and allowed (hundreds), turnovers lost and forced

$$\hat{y} = xW = \begin{bmatrix} 3.76 & 2.88 & 1 & 2 \end{bmatrix} \begin{bmatrix} -1 & 1 \\ 1 & -1 \\ 0.5 & -0.5 \\ -0.5 & 0.5 \end{bmatrix} = \begin{bmatrix} -1.38 & 1.38 \end{bmatrix}$$

the second column gives the win score

$$3.76(1) + 2.88(-1) + 1(-0.5) + 2(0.5) = 1.38$$

The second column says every hundred yards gained counts toward a win, every hundred allowed counts against, and a turnover is worth about fifty yards. The first column is the same with the signs flipped, so it scores a loss. Purdue's win score is the bigger one, so this matrix calls it a win (correctly).

Matrix Multiplication in Code

In PyTorch the whole of the last page is one `@`. In code we write `W` for W (it's easier to type), and `y_hat` for the output:

```python
import torch

# Purdue 2012: yards (in hundreds) and turnovers
x = torch.tensor([[3.76, 2.88, 1., 2.]])
W = torch.tensor([[-1.0,  1.0],
                  [ 1.0, -1.0],
                  [ 0.5, -0.5],
                  [-0.5,  0.5]])
y_hat = x @ W
print(x.shape, W.shape, y_hat.shape)
print(y_hat)
```

OUTPUT

```
torch.Size([1, 4]) torch.Size([4, 2]) torch.Size([1, 2])
tensor([[-1.3800,  1.3800]])
```

The shapes are the tiles from the last page, `[1, 4]` at `[4, 2]` gives `[1, 2]`. The double brackets in x make it a matrix with one row, rather than a plain list of four numbers, so its shape matches the picture.

If the inner numbers don't match, PyTorch refuses (try `W @ x`, which asks a `[4, 2]` to meet a `[1, 4]`). Most of the errors you'll see while building irishQwen are some version of that complaint, so it's worth getting used to checking shapes.

Every Game at Once

Nothing in the rule says the first matrix can only have one row. If we stack all `141` games into a matrix `X` of shape `<141, 4>`, then `X @ W` is `<141, 2>`: every game's loss score and win score in a single multiplication. For each game we pick the column with the bigger score (`argmax` gives its position, `0` for loss and `1` for win), and compare:

```
import pandas as pd
import torch

games = pd.read_csv("nd_football.csv")
yards = games[["nd_total_yards", "opp_total_yards"]]
turnovers = games[["nd_turnovers", "opp_turnovers"]]
X = torch.tensor(pd.concat([yards / 100, turnovers],
                           axis=1).to_numpy(),
                 dtype=torch.float32)
# column 0 is the loss score, column 1 the win score
y = torch.tensor(games["win"].to_numpy())

W = torch.tensor([[-1.0,  1.0],
                  [ 1.0, -1.0],
                  [ 0.5, -0.5],
                  [-0.5,  0.5]])
pred = torch.argmax(X @ W, dim=1)
print(X.shape, (X @ W).shape)
print((pred == y).sum().item(), "of", len(y))
```

OUTPUT

```
torch.Size([141, 4]) torch.Size([141, 2])
125 of 141
```

Our hand-picked matrix gets `125` of `141` games right. That is better than the laziest possible model, guessing "win" every time, which gets `106`.

Input Space and Output Space

Before the multiplication, each game was a point in a 4-dimensional space of yards and turnovers, the **input space**. After it, each game is a point in a 2-dimensional space whose axes are "how much this looks like a win" and "how much this looks like a loss". A space where the numbers are the answer is called the **output space**.

So **a model is a way of moving data from input space to output space**, and matrix multiplication is how it moves. Real models do this many times in a row, each matrix turning one representation into a (hopefully) more useful one, and the last one landing in the space of answers.

ChatGPT ends the same way. Inside irishQwen every piece of a word is a vector of 1,024 numbers, and its very last step is a single matrix of shape <1024, 151936>. That gives one score for every one of the 151,936 pieces of words it knows, and the biggest score is its guess at what comes next. It's our win score and loss score, just with a much longer list of possible answers.

Model Weights and Biases

The numbers in W decide how much each feature counts toward each output, and they are called the model's **weights**. Most models also add one more number to each output after the multiplication, called a **bias**, which shifts a score up or down no matter what the game looked like:

```
[1.7]
```

inputs at the weights, plus the bias

$$\hat{y} = xW + b$$

We picked our weights by hand, which works for `8` of them and a sport we understand. Unfortunately, irishQwen has around `600` million, and nobody can pick those. Real models start from random weights instead, and let training find better ones.

Random Weights

Here is our layer again on every game, with the weights drawn at random and the bias set to zero:

```
import pandas as pd
import torch

games = pd.read_csv("nd_football.csv")
yards = games[["nd_total_yards", "opp_total_yards"]]
turnovers = games[["nd_turnovers", "opp_turnovers"]]
X = torch.tensor(pd.concat([yards / 100, turnovers],
                           axis=1).to_numpy(),
                 dtype=torch.float32)
y = torch.tensor(games["win"].to_numpy())

torch.manual_seed(0)
W = 0.01 * torch.randn(4, 2)
b = torch.zeros(2)
pred = torch.argmax(X @ W + b, dim=1)
print(W)
print((pred == y).sum().item(), "of", len(y))
```

OUTPUT

```
tensor([[ 0.0013, -0.0013],
        [ 0.0064,  0.0010],
        [-0.0054,  0.0036],
        [ 0.0130,  0.0095]])
33 of 141
```

`randn` draws random numbers, which we shrink to around `0.01`, and the bias starts at zero. `33` right is far worse than guessing "win" every time, which gets `106`. (Your numbers will differ with a different seed, or on desktop PyTorch, but a random matrix has no reason to beat that guess.)

Linear Layers

A matrix of weights plus a bias, $xW + b$, is used so often that PyTorch has it ready made. It's called a **linear layer**, and `torch.nn.Linear(4, 2)` is one that takes in `4` features and gives back `2`, with its weights already set to small random numbers:

```python
import torch

layer = torch.nn.Linear(4, 2)
print(layer.weight.shape, layer.bias.shape)

x = torch.tensor([[3.76, 2.88, 1., 2.]])
print(layer(x).shape)
```

OUTPUT

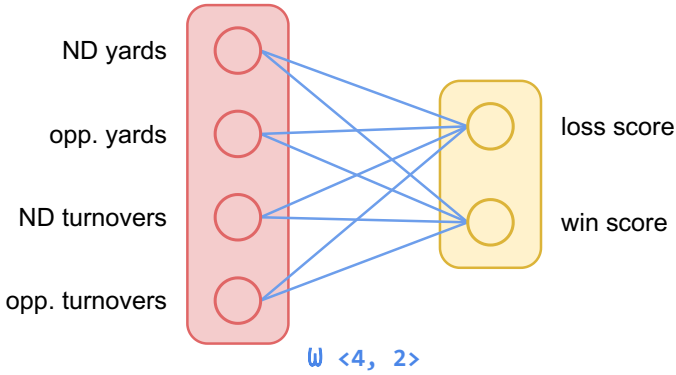
```
torch.Size([2, 4]) torch.Size([2])
torch.Size([1, 2])
```

Notice that PyTorch stores the weights the other way round, as `[2, 4]` (one row per output), and flips them before multiplying. That's a choice of bookkeeping, not a different layer. Calling `layer(x)` does the multiplication and adds the bias, and Purdue comes out as two numbers, as before.

This one line is the same building block irishQwen is made of. Its last layer, the one from Input Space and Output Space, is a linear layer from `1,024` numbers to `151,936`, and most of what comes before it is linear layers too. What comes out of a linear layer is a list of scores, not probabilities, and turning scores into probabilities is the next chapter's job.

Model Architecture

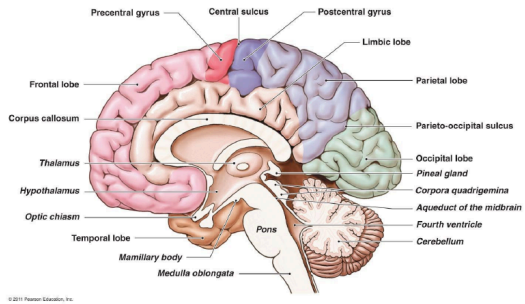
Our model so far is one linear layer, four inputs joined to two outputs by eight weights:

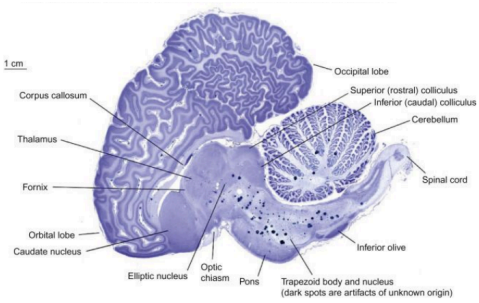


Which matrices a model uses, how big they are and in what order the data goes through them is decided before any learning happens. The specific set of matrix multiplications the data goes through in a model is called its **architecture**. GPT-3 sends every piece of a prompt through **96** repeated blocks, each a handful of matrix multiplications, and irishQwen has **28**.

Architecture in the Brain

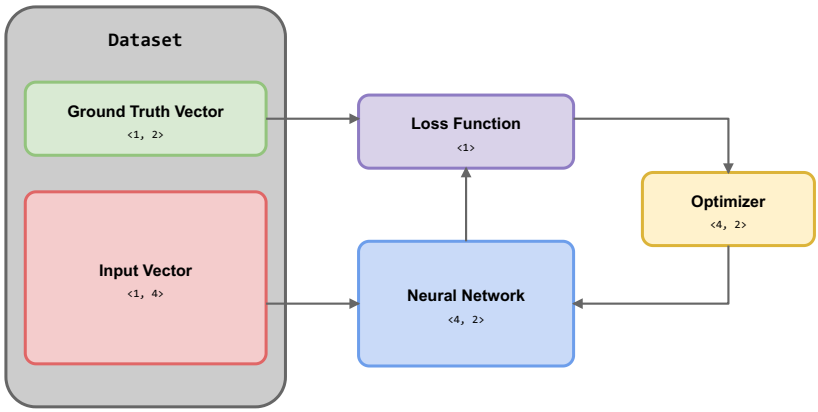
Brains make a decent comparison. A human brain and a dolphin brain (pictured below) are built quite differently, and that is one of the major ways we differ from other animals. Yet all people share the same architecture, and people are still wildly different, because **what each brain has learned is different**. Two models with the same architecture and different weights make different predictions, which is exactly what we just saw: same layer, our hand-picked weights got `125` games right and the random ones `108`.





Training a Model

So our model with random weights is bad. To improve it we can **train** it: let it practice over and over on the same games, changing its weights a little each time so that its predictions get better. Everything the model learns lives in its weights, so **training a model means updating its weights**.



Pictured above is how that goes. The inputs go through the network, its predictions are compared with the ground truth, and an optimizer uses that comparison to decide how to change the weights. Then the whole thing goes around again.

Measuring Mistakes

The comparison is the piece we're missing. For Purdue our hand-picked model gave a loss score of `-1.38` and a win score of `1.38`, and the truth was `[0, 1]`. The bigger score is in the right place, which is good, but "in the right place" isn't something we can hand to a computer.

Counting the games we get right isn't enough. A model whose win score for Purdue is just barely bigger than its loss score and one where it wins by `2.76` are both "right", but the second is clearly more sure of itself, and a small change to the weights almost never changes which games are right at all. So a count can't tell us which small changes are improvements.

What we need is **a single number that says how wrong a prediction is**, smaller when the prediction is closer to the truth, so that changing the weights to make it smaller makes the model better. A function that gives us that number is called a **loss function**, and it is the next thing we need to build.